Ujian Tengah Semester

Perbandingan Metode Konvensional dan Deep Learning pada Kasus Titanic, Digit Recognizer, dan Disaster Tweets

Nama	**:**	**Fajar Jauza Maylana**
NIM	**:**	**1202220123**
Kelas	**:**	**SI-46-EDM**
Tanggal Pengumpulan	**:**	**28 April 2026**
Link GitHub Repository	**:**	https://github.com/Fajarjauza/uts-deep-learning.git

PENDAHULUAN

1. Latar Belakang

Kemajuan dalam *machine learning* dan *deep learning* telah menawarkan berbagai strategi untuk mengatasi masalah klasifikasi pada data tabular, gambar, dan teks. Metode tradisional seperti Logistic Regression, Random Forest, Support Vector Machine, K-Nearest Neighbors, dan Naive Bayes tetap banyak digunakan karena sifatnya yang cukup sederhana, efisien, serta mudah dipahami. Penerapan metode-metode tersebut didukung oleh pustaka seperti Scikit-learn yang menyediakan beragam algoritma machine learning untuk pembelajaran terawasi dan tidak terawasi dalam bahasa Python (Pedregosa et al., 2011). Di sisi lain, deep learning memungkinkan model untuk secara otomatis mempelajari representasi fitur melalui sejumlah lapisan pemrosesan, sehingga banyak diterapkan pada masalah dengan pola data yang rumit seperti gambar dan teks (Alzubaidi et al., 2021).

Walau demikian, deep learning tidak selalu merupakan metode yang paling baik untuk semua jenis dataset. Kinerjanya sangat dipengaruhi oleh jumlah data, kompleksitas pola, mutu preprocessing, serta kapasitas komputasi yang tersedia. Pada dataset kecil atau data tabular yang sederhana, metode tradisional masih dapat bersaing bahkan seringkali lebih unggul dibandingkan model deep learning karena lebih stabil dan kurang rentan terhadap overfitting. Sebaliknya, pada data gambar, Convolutional Neural Network (CNN) umumnya lebih efektif karena mampu menangkap pola spasial seperti garis, tepi, lengkungan, dan bentuk objek melalui operasi konvolusi dan pooling. Konsep lapisan konvolusi dan pooling banyak diterapkan dalam arsitektur CNN untuk tugas penglihatan komputer (Stanford CS231, 2026). Sedangkan pada data teks, Long Short-Term Memory (LSTM) dapat digunakan untuk mempelajari ketergantungan antar kata dalam urutan teks sebab dirancang untuk menangani masalah ketergantungan jangka panjang pada data berurutan (Hochreiter & Schmidhuber, 1997).

Dokumen ini bertujuan untuk membandingkan kinerja metode tradisional dan deep learning pada tiga kasus dengan karakteristik data yang berbeda. Kasus pertama adalah klasifikasi keselamatan penumpang Titanic yang mewakili data tabular. Di sini, Logistic Regression dan Random Forest dibandingkan dengan Multilayer Perceptron (MLP). Kasus kedua adalah klasifikasi digit tulisan tangan menggunakan dataset Digit Recognizer/MNIST yang mewakili data gambar. Dalam hal ini, pendekatan HOG + SVM dan PCA + KNN dibandingkan dengan CNN. Kasus ketiga adalah klasifikasi tweet bencana menggunakan dataset Natural Language Processing with Disaster Tweets yang mewakili data teks. Di sini, pendekatan TF-IDF dengan Logistic Regression, Linear SVM, dan Multinomial Naive Bayes dibandingkan dengan model berbasis Embedding dan LSTM.

Seluruh eksperimen dilakukan dengan seed acak 42 dan pembagian data pelatihan-validasi-penguji yang konsisten di setiap kasus agar perbandingan antar model berlangsung adil dan dapat direproduksi. Evaluasi dilakukan menggunakan metrik akurasi, presisi, recall, F1-score, waktu pelatihan, waktu inferensi, serta jumlah parameter atau kompleksitas model. Selain itu, laporan ini juga menyajikan analisis kesalahan untuk memahami pola kesalahan model, serta diskusi lintas kasus untuk menilai pada tipe data mana deep learning memberikan peningkatan yang signifikan dan dalam kondisi apa metode tradisional masih lebih efektif.

PEMBAHASAN

1. Kasus 1: Titanic - Tabular Classification

a. Deskripsi Dataset

Dataset Titanic merupakan dataset klasifikasi biner yang digunakan untuk memprediksi apakah seorang penumpang selamat atau tidak berdasarkan informasi penumpang. Target pada dataset ini adalah Survived, dengan nilai 0 untuk penumpang yang tidak selamat dan 1 untuk penumpang yang selamat. Dataset ini berbentuk data tabular dengan kombinasi fitur numerik dan kategorikal, seperti Pclass, Sex, Age, SibSp, Parch, Fare, Embarked, Name, dan Cabin. Tantangan utama pada dataset ini adalah jumlah data training yang relatif kecil, yaitu sekitar 891 baris, adanya missing value pada beberapa fitur seperti Age, Cabin, dan Embarked, serta perlunya feature engineering agar informasi tambahan seperti gelar penumpang, ukuran keluarga, dan keberadaan data kabin dapat

b. Eksplorasi Data

Eksplorasi data dilakukan untuk memahami distribusi target dan hubungan beberapa fitur penting terhadap peluang survival. Visualisasi yang digunakan antara lain distribusi kelas Survived serta perbandingan survival berdasarkan Sex dan Pclass. Hasil EDA menunjukkan bahwa jumlah penumpang yang tidak selamat lebih banyak dibanding penumpang yang selamat. Selain itu, fitur Sex dan Pclass memiliki hubungan yang cukup kuat dengan peluang survival. Penumpang perempuan dan penumpang dari kelas yang lebih tinggi cenderung memiliki peluang selamat lebih besar dibanding kelompok lainnya.

c. Preprocessing

Preprocessing dilakukan untuk mengubah data mentah menjadi format numerik yang dapat diproses oleh model. Missing value pada fitur numerik dan kategorikal diisi menggunakan strategi imputasi. Beberapa fitur baru dibuat untuk menangkap informasi tambahan dari data asli.

Langkah Preprocessing	Alasan
Imputasi Age	Mengatasi missing value pada umur
Imputasi Embarked	Mengisi nilai kategorikal yang kosong
Imputasi Fare	Mengatasi kemungkinan nilai kosong pada data test
Ekstraksi Title dari Name	Mengambil informasi sosial/status dari nama penumpang
Membuat FamilySize	Menggabungkan informasi SibSp dan Parch
Membuat IsAlone	Menandai apakah penumpang bepergian sendiri
Membuat HasCabin	Mengubah informasi kabin menjadi indikator biner
Binning Age dan Fare	Mengelompokkan nilai kontinu agar pola lebih mudah ditangkap
One-hot encoding	Mengubah fitur kategorikal menjadi numerik
Scaling fitur numerik	Membantu Logistic Regression dan MLP berlatih lebih stabil
Split data dengan seed 42	Menjaga fairness dan reproducibility

d. Metode Konvensional

Metode konvensional yang digunakan pada kasus Titanic adalah Logistic Regression dan Random Forest. Logistic Regression digunakan sebagai baseline linear untuk klasifikasi biner, sedangkan Random Forest digunakan untuk menangkap hubungan non-linear antar fitur melalui ensemble decision tree.

Model	Karakteristik	Hyperparameter Utama
Logistic Regression	Model linear untuk klasifikasi biner	max_iter, random_state=42
Random Forest	Ensemble decision tree	n_estimators, max_depth, random_state=42

Hasil metode konvensional:

Model	Accuracy	Precision	Recall	F1-score	Training Time
Logistic Regression	0.8436	0.7971	0.7971	0.7971	0.1235 s
Random Forest	0.7989	0.7200	0.7826	0.7500	0.5928 s

Berdasarkan hasil tersebut, Logistic Regression memberikan performa terbaik di antara metode konvensional dengan F1-score sebesar 0.7971. Hal ini menunjukkan bahwa setelah preprocessing dan feature engineering dilakukan, pola pada dataset Titanic masih dapat ditangkap dengan baik oleh model linear.

e. Metode Deep Learning

Metode deep learning yang digunakan adalah Multilayer Perceptron (MLP). Model ini terdiri dari beberapa dense layer yang dikombinasikan dengan Batch Normalization, Dropout, L2 regularization, dan EarlyStopping untuk mengurangi risiko overfitting. Optimizer yang digunakan adalah Adam, dengan output sigmoid karena target klasifikasi bersifat biner. MLP dipilih karena dataset Titanic merupakan data tabular yang setelah preprocessing berubah menjadi kumpulan fitur numerik. Dense layer pada MLP dapat mempelajari hubungan non-linear antar fitur. Namun, karena jumlah data relatif kecil, MLP berisiko mengalami overfitting dan membutuhkan waktu training lebih lama dibanding model konvensional.

Hasil metode deep learning:

Model	Accuracy	Precision	Recall	F1-score	Training Time	Jumlah Parameter
MLP Deep Learning	0.7989	0.7619	0.6957	0.7273	16.1387 s	4.417

f. Perbandingan Hasil

Model	Jenis Metode	Accuracy	F1-score	Training Time	Jumlah Parameter / Kompleksitas
Logistic Regression	Konvensional	0.8436	0.7971	0.1235 s	Rendah
Random Forest	Konvensional	0.7989	0.7500	0.5928 s	Sedang
MLP	Deep Learning	0.7989	0.7273	16.1387 s	4.417 parameter

Berdasarkan hasil eksperimen, Logistic Regression menjadi model terbaik pada kasus Titanic dengan F1-score sebesar 0.7971. MLP membutuhkan waktu training paling lama, yaitu 16.1387 detik, tetapi tidak memberikan peningkatan performa dibanding metode

konvensional. Hasil ini menunjukkan bahwa pada dataset tabular kecil, metode konvensional masih sangat kompetitif, lebih efisien secara komputasi, dan lebih mudah diinterpretasikan dibanding deep learning.

g. Analisis Error

Analisis error dilakukan dengan melihat contoh data yang salah diklasifikasikan oleh model. Kesalahan prediksi pada Titanic umumnya terjadi pada penumpang dengan karakteristik yang tidak mengikuti pola mayoritas.

No	Karakteristik Data	Label Asli	Prediksi	Interpretasi
1	Penumpang perempuan dari kelas rendah	0	1	Model dapat terlalu mengandalkan pola bahwa perempuan memiliki peluang survival tinggi
2	Penumpang laki-laki dari kelas tinggi	1	0	Model dapat terlalu mengandalkan pola bahwa laki-laki memiliki peluang survival rendah
3	Penumpang dengan umur atau fare hasil imputasi/binning	1	0	Informasi hasil imputasi atau pengelompokan dapat mengurangi detail data asli

Analisis error dilakukan dengan melihat contoh data yang salah diklasifikasikan oleh model. Kesalahan prediksi pada Titanic umumnya terjadi pada penumpang dengan karakteristik yang tidak mengikuti pola mayoritas.

2. Kasus 2: Digit Recognizer - Image Classification

a. Deskripsi Dataset

Dataset Digit Recognizer merupakan dataset klasifikasi multi-kelas yang bertujuan mengenali digit tulisan tangan dari angka 0 sampai 9. Setiap data berupa gambar grayscale berukuran 28×28 piksel. Jika gambar diratakan, setiap sampel memiliki 784 fitur piksel. Tantangan utama pada dataset ini adalah variasi bentuk tulisan tangan antar individu serta kemiripan visual antar beberapa digit. Beberapa digit seperti 3 dan 5, 4 dan 9, serta 1 dan 7 dapat memiliki bentuk yang mirip sehingga berpotensi menyebabkan kesalahan klasifikasi.

b. Eksplorasi Data

Eksplorasi data dilakukan dengan menampilkan distribusi label digit 0–9 dan contoh gambar dari masing-masing digit. Visualisasi distribusi label digunakan untuk melihat keseimbangan jumlah data antar kelas, sedangkan visualisasi contoh gambar digunakan untuk memahami variasi bentuk tulisan tangan. Hasil EDA menunjukkan bahwa jumlah data pada setiap kelas digit relatif seimbang. Contoh gambar memperlihatkan bahwa beberapa digit memiliki bentuk yang jelas, tetapi ada juga gambar dengan bentuk ambigu atau mirip dengan kelas lain.

c. Preprocessing

Preprocessing dilakukan dengan menyesuaikan format data untuk model konvensional dan CNN.

Langkah Preprocessing	Alasan

Normalisasi pixel /255.0	Mengubah nilai pixel ke rentang 0–1 agar training stabil
Reshape ke 28×28×1	Menyesuaikan input CNN dan mempertahankan struktur spasial
Flatten image	Menyesuaikan input untuk metode konvensional
Ekstraksi HOG	Mengambil fitur bentuk, tepi, dan arah gradien untuk SVM
PCA	Mengurangi dimensi fitur sebelum digunakan oleh KNN
Encoding label	Menyesuaikan format output klasifikasi multi-kelas
Split data dengan seed 42	Menjaga split konsisten dan reproducible

d. Metode Konvensional

Metode konvensional yang digunakan adalah HOG + SVM dan PCA + KNN. HOG digunakan untuk mengekstraksi pola bentuk dan arah gradien dari gambar, kemudian diklasifikasikan menggunakan SVM. PCA digunakan untuk mereduksi dimensi fitur piksel sebelum diklasifikasikan menggunakan KNN.

Model	Feature Engineering	Hyperparameter Utama
HOG + SVM	HOG	Kernel/C, random_state=42
PCA + KNN	PCA	Jumlah komponen PCA, n_neighbors

Hasil metode konvensional:

Model	Accuracy	Training Time	Inference Time	Misclassification Count
HOG + SVM	0.9770	13.9087 s	0.0426 s	193
PCA + KNN	0.9701	1.1828 s	3.0732 s	251

HOG + SVM menghasilkan akurasi lebih tinggi dibanding PCA + KNN. Hal ini menunjukkan bahwa fitur HOG cukup efektif dalam merepresentasikan bentuk digit tulisan tangan. PCA + KNN memiliki waktu training lebih cepat, tetapi waktu inferensinya lebih lambat karena KNN perlu membandingkan data baru dengan data training.

e. Metode Deep Learning

Metode deep learning yang digunakan adalah Convolutional Neural Network (CNN). Arsitektur CNN terdiri dari beberapa convolution layer, max pooling layer, flatten layer, dense layer, dropout, dan output softmax untuk klasifikasi 10 kelas digit. CNN dipilih karena data berupa gambar memiliki struktur spasial. Convolution layer mampu menangkap pola lokal seperti garis, tepi, lengkungan, dan sudut yang penting untuk membedakan digit tulisan tangan. Pooling layer membantu mereduksi dimensi fitur sekaligus mempertahankan informasi penting, sedangkan dense layer digunakan untuk klasifikasi akhir.

Hasil metode deep learning:

Model	Accuracy	Training Time	Inference Time	Misclassification Count
CNN	0.9908	309.4485 s	2.2559 s	77

f. Perbandingan Hasil

Model	Jenis Metode	Accuracy	Training Time	Inference Time	Misclassification Count
CNN	Deep Learning	0.9908	309.4485 s	2.2559 s	77

HOG + SVM	Konvensional	0.9770	13.9087 s	0.0426 s	193
PCA + KNN	Konvensional	0.9701	1.1828 s	3.0732 s	251

CNN memberikan performa terbaik dengan accuracy sebesar 0.9908 dan jumlah kesalahan paling sedikit, yaitu 77 gambar. Hal ini menunjukkan bahwa deep learning memberikan peningkatan signifikan pada data citra karena CNN mampu mempelajari fitur visual secara otomatis. Namun, CNN memiliki waktu training paling lama, sehingga terdapat trade-off antara peningkatan akurasi dan biaya komputasi.

g. Analisis Error

Kesalahan klasifikasi pada dataset digit umumnya terjadi pada gambar dengan bentuk tulisan yang ambigu atau menyerupai kelas lain.

No	Label Asli	Prediksi	Interpretasi
1	4	9	Bentuk digit memiliki loop atau lengkungan yang menyerupai angka 9
2	3	5	Pola lengkungan antara angka 3 dan 5 terlihat mirip
3	7	1	Garis horizontal atau kemiringan digit kurang jelas sehingga menyerupai angka 1

Meskipun CNN memiliki performa terbaik, model tetap dapat melakukan kesalahan ketika bentuk digit tidak jelas, terlalu tipis, terlalu miring, atau memiliki kemiripan visual dengan kelas lain.

3. Kasus 3: Disaster Tweets - Text Classification

a. Deskripsi Dataset

Dataset Disaster Tweets merupakan dataset klasifikasi biner yang bertujuan menentukan apakah sebuah tweet benar-benar membahas bencana atau tidak. Target bernilai 1 untuk tweet disaster dan 0 untuk tweet non-disaster. Fitur utama pada dataset ini adalah text, dengan fitur tambahan seperti keyword dan location. Tantangan utama pada dataset ini adalah teks yang pendek, penggunaan bahasa informal, adanya URL, mention, hashtag, simbol, serta kata-kata ambigu yang dapat digunakan secara literal maupun metaforis. Kata seperti “fire”, “crash”, “storm”, atau “burning” dapat muncul pada tweet bencana maupun non-bencana sehingga konteks menjadi penting.

b. Eksplorasi Data

Eksplorasi data dilakukan dengan menampilkan distribusi label disaster dan non-disaster, serta analisis sederhana terhadap karakteristik teks seperti panjang tweet atau kata yang sering muncul. Hasil EDA menunjukkan bahwa data tidak sepenuhnya seimbang, dengan jumlah tweet non-disaster lebih banyak dibanding tweet disaster. Selain itu, beberapa kata bernuansa bencana dapat muncul pada kedua kelas, sehingga model perlu membedakan konteks penggunaan kata, bukan hanya keberadaan kata tertentu.

c. Preprocessing

Preprocessing teks dilakukan untuk mengurangi noise dan membuat data lebih konsisten sebelum dimasukkan ke model.

Langkah Preprocessing	Alasan
Lowercase	Menyamakan format huruf
Menghapus URL	URL biasanya tidak memberi informasi langsung terhadap kelas
Menghapus mention	Mengurangi noise dari username
Menghapus simbol hashtag	Membersihkan token
Menghapus karakter non-huruf	Mengurangi noise
Menghapus spasi berlebih	Merapikan teks
TF-IDF	Representasi teks untuk model konvensional
Tokenizer	Mengubah teks menjadi sequence angka untuk LSTM
Padding	Menyamakan panjang sequence input LSTM
Split data dengan seed 42	Menjaga fairness dan reproducibility

d. Metode Konvensional

Metode konvensional yang digunakan adalah TF-IDF dengan Logistic Regression, Linear SVM, dan Multinomial Naive Bayes. TF-IDF digunakan untuk merepresentasikan teks berdasarkan bobot kemunculan kata, sedangkan model classifier digunakan untuk memprediksi apakah tweet termasuk disaster atau non-disaster.

Model	Representasi	Hyperparameter
Logistic Regression	TF-IDF Unigram	C = 0.1, 1, 10
Linear SVM	TF-IDF Unigram-Bigram	C = 0.1, 1, 10
Multinomial Naive Bayes	TF-IDF Unigram-Bigram	alpha = 0.1, 0.5, 1.0

Hasil metode konvensional:

Model	Accuracy	Precision	Recall	F1-score	Training Time	Inference Time
TF-IDF + Logistic Regression	0.8188	0.8387	0.7156	0.7723	0.1380 s	0.0035 s
TF-IDF + Linear SVM	0.7984	0.7785	0.7416	0.7596	0.1796 s	0.0065 s
TF-IDF + Multinomial NB	0.8188	0.8810	0.6682	0.7600	0.0210 s	0.0039 s

Logistic Regression dengan TF-IDF menghasilkan F1-score tertinggi sebesar 0.7723. Linear SVM memiliki recall yang cukup baik, sedangkan Multinomial Naive Bayes memiliki precision tertinggi tetapi recall lebih rendah. Hal ini menunjukkan bahwa TF-IDF masih menjadi representasi yang kuat untuk klasifikasi teks pendek seperti tweet.

e. Metode Deep Learning

Metode deep learning yang digunakan adalah model berbasis Embedding dan LSTM. Embedding layer digunakan untuk mengubah token menjadi representasi vektor, LSTM digunakan untuk mempelajari pola urutan kata, dan dense layer dengan aktivasi sigmoid digunakan untuk klasifikasi biner. LSTM dipilih karena data teks memiliki bentuk sekuensial, sehingga urutan kata dapat memberikan informasi penting dalam menentukan makna tweet. Namun, pada eksperimen ini LSTM tidak berhasil mengungguli metode konvensional. Model menghasilkan F1-score 0 karena cenderung memprediksi seluruh data validation sebagai kelas non-disaster. Hal ini dapat disebabkan oleh ukuran dataset yang

terbatas, tweet yang pendek dan noisy, distribusi kelas yang tidak sepenuhnya seimbang, serta konfigurasi LSTM yang belum optimal.

Hasil metode deep learning:

Model	Accuracy	Precision	Recall	F1-score	Training Time	Inference Time
Embedding + LSTM	0.5706	0.0000	0.0000	0.0000	128.7603 s	1.1069 s

f. Perbandingan Hasil

Model	Jenis Metode	Accuracy	Precision	Recall	F1-score	Training Time	Inference Time
TF-IDF + Logistic Regression	Konvensional	0.8188	0.8387	0.7156	0.7723	0.1380 s	0.0035 s
TF-IDF + Multinomial NB	Konvensional	0.8188	0.8810	0.6682	0.7600	0.0210 s	0.0039 s
TF-IDF + Linear SVM	Konvensional	0.7984	0.7785	0.7416	0.7596	0.1796 s	0.0065 s
Embedding + LSTM	Deep Learning	0.5706	0.0000	0.0000	0.0000	128.7603 s	1.1069 s

Model terbaik pada kasus Disaster Tweets adalah TF-IDF + Logistic Regression dengan F1-score sebesar 0.7723. Model ini jauh lebih cepat dibanding LSTM dan memiliki performa yang lebih stabil. Hasil ini menunjukkan bahwa pada teks pendek seperti tweet, metode konvensional berbasis TF-IDF masih sangat kompetitif. Sebaliknya, LSTM membutuhkan data yang lebih besar, tuning yang lebih hati-hati, dan strategi tambahan seperti class weighting atau threshold tuning agar dapat bekerja optimal.

g. Analisis Error

Kesalahan pada model teks banyak terjadi karena adanya ambiguitas kata dan konteks yang sulit ditangkap dari tweet pendek.

No	Tweet	Label Asli	Prediksi	Interpretasi
1	Tweet non-disaster yang mengandung kata seperti “fire” atau “crash”	0	1	Kata bernuansa bencana digunakan secara metaforis sehingga model salah menganggapnya sebagai disaster
2	Tweet disaster tanpa kata bencana eksplisit	1	0	Konteks bencana tidak cukup kuat setelah preprocessing
3	Tweet pendek dengan informasi terbatas	1	0	Model kesulitan memahami konteks karena jumlah token sedikit

Pada LSTM, error lebih dominan karena model cenderung memprediksi kelas non-disaster untuk hampir seluruh data validation. Sementara itu, pada model TF-IDF, kesalahan umumnya disebabkan oleh kata-kata ambigu atau konteks tweet yang tidak cukup eksplisit.

DISKUSI LINTAS KASUS

Berdasarkan hasil eksperimen pada tiga kasus, terlihat bahwa efektivitas deep learning sangat bergantung pada karakteristik data yang digunakan. Deep learning tidak selalu memberikan performa terbaik pada semua tipe data. Pada data citra, deep learning memberikan peningkatan paling signifikan. Namun, pada data tabular kecil dan teks pendek, metode konvensional masih sangat kompetitif bahkan mampu mengungguli model deep learning.

Pada Kasus 1, yaitu Titanic, model terbaik diperoleh oleh Logistic Regression dengan F1-score sebesar 0.7971 dan accuracy sebesar 0.8436. Sementara itu, MLP sebagai model deep learning memperoleh F1-score sebesar 0.7273 dan accuracy sebesar 0.7989. Hasil ini menunjukkan bahwa pada dataset tabular dengan jumlah data relatif kecil, metode konvensional masih lebih efektif. Dataset Titanic hanya memiliki sekitar 891 data training dan fitur-fiturnya sudah cukup informatif setelah dilakukan preprocessing dan feature engineering. Model seperti Logistic Regression dapat menangkap hubungan antar fitur dengan baik, lebih cepat dilatih, dan lebih mudah diinterpretasikan. Sebaliknya, MLP membutuhkan waktu training lebih lama dan memiliki risiko overfitting lebih tinggi karena jumlah data terbatas.

Pada Kasus 2, yaitu Digit Recognizer, deep learning memberikan peningkatan performa yang paling jelas. CNN memperoleh accuracy sebesar 0.9908 dengan jumlah kesalahan 77 gambar, lebih baik dibanding HOG + SVM yang memperoleh accuracy 0.9770 dengan 193 kesalahan, serta PCA + KNN yang memperoleh accuracy 0.9701 dengan 251 kesalahan. Hasil ini menunjukkan bahwa CNN sangat sesuai untuk data citra karena mampu mempelajari fitur spasial secara otomatis, seperti garis, tepi, lengkungan, dan bentuk lokal pada digit. Berbeda dengan metode konvensional yang membutuhkan ekstraksi fitur manual seperti HOG atau reduksi dimensi seperti PCA, CNN dapat belajar langsung dari data piksel sehingga representasi fitur yang diperoleh lebih relevan untuk klasifikasi gambar.

Pada Kasus 3, yaitu Disaster Tweets, metode konvensional kembali menunjukkan performa yang lebih baik. TF-IDF + Logistic Regression menghasilkan F1-score tertinggi sebesar 0.7723 dan accuracy sebesar 0.8188. Sebaliknya, model Embedding + LSTM hanya memperoleh accuracy sebesar 0.5706 dan F1-score 0.0000. Hasil ini menunjukkan bahwa pada data teks pendek seperti tweet, metode konvensional berbasis TF-IDF masih sangat kuat. Tweet memiliki panjang terbatas, banyak noise seperti URL, mention, hashtag, serta mengandung kata-kata ambigu yang dapat digunakan secara literal maupun metaforis. Pada kondisi tersebut, TF-IDF mampu menangkap kata-kata diskriminatif secara sederhana dan efektif. Sementara itu, LSTM membutuhkan data yang lebih besar, preprocessing yang lebih hati-hati, serta tuning tambahan seperti class weighting, threshold adjustment, atau penggunaan embedding pretrained agar dapat bekerja optimal.

Secara umum, deep learning memberikan peningkatan signifikan terutama pada data citra. Hal ini terlihat pada kasus Digit Recognizer, di mana CNN mengungguli seluruh metode konvensional. Keunggulan CNN berasal dari kemampuannya mempertahankan struktur spasial gambar dan mempelajari pola visual secara otomatis. Dengan demikian, deep learning sangat sesuai untuk tipe data yang memiliki struktur kompleks dan pola lokal yang sulit dirancang secara manual. Sebaliknya, metode konvensional masih sangat kompetitif pada data tabular kecil dan teks pendek. Pada Titanic, Logistic Regression lebih unggul karena dataset kecil dan fitur hasil preprocessing sudah cukup representatif. Pada Disaster Tweets, TF-IDF + Logistic Regression lebih unggul karena teks pendek dapat direpresentasikan secara efektif menggunakan bobot kata. Hal ini menunjukkan bahwa model yang lebih kompleks tidak selalu menghasilkan performa lebih baik, terutama jika jumlah data terbatas atau sinyal dalam data sudah dapat ditangkap oleh metode sederhana.

Trade-off utama antara metode konvensional dan deep learning terlihat pada tiga aspek, yaitu akurasi, biaya komputasi, dan interpretabilitas. Dari sisi akurasi, deep learning unggul pada data citra, tetapi tidak unggul pada Titanic dan Disaster Tweets. Dari sisi biaya komputasi, metode konvensional jauh lebih efisien. Logistic Regression pada Titanic hanya membutuhkan waktu training sekitar 0.1235 detik, sedangkan MLP membutuhkan 16.1387 detik. Pada Disaster Tweets, TF-IDF + Logistic Regression hanya membutuhkan waktu training sekitar 0.1380 detik, sedangkan LSTM membutuhkan 128.7603 detik. Pada Digit Recognizer, CNN memang menghasilkan akurasi tertinggi, tetapi waktu training-nya mencapai 309.4485 detik, jauh lebih lama dibanding HOG + SVM dan PCA + KNN. Dari sisi interpretabilitas, metode konvensional juga lebih mudah dijelaskan. Logistic Regression dapat dianalisis melalui bobot fitur, Random Forest dapat dianalisis melalui feature importance, dan TF-IDF memungkinkan analisis kata-kata yang berkontribusi terhadap prediksi. Sebaliknya, model deep learning seperti MLP, CNN, dan LSTM memiliki representasi internal yang lebih kompleks sehingga lebih sulit diinterpretasikan secara langsung. Meskipun CNN dapat dianalisis dengan teknik visualisasi fitur, interpretasinya tetap lebih kompleks dibanding model konvensional.

Berdasarkan ketiga kasus tersebut, dapat disimpulkan bahwa pemilihan model harus mempertimbangkan karakteristik data, jumlah data, tujuan analisis, dan sumber daya komputasi. Deep learning lebih cocok digunakan ketika data memiliki struktur kompleks seperti citra dan tersedia cukup data serta komputasi. Metode konvensional tetap menjadi pilihan kuat untuk dataset kecil, data tabular, dan teks pendek, terutama ketika efisiensi dan interpretabilitas menjadi pertimbangan penting.

Ringkasan Perbandingan Lintas Kasus

Kasus	**Tipe Data**	**Model Terbaik**	**Metode Terbaik**	**Kesimpulan**
Titanic	Tabular	Logistic Regression	Konvensional	Data kecil dan fitur tabular membuat model konvensional lebih stabil
Digit Recognizer	Citra	CNN	Deep Learning	CNN unggul karena mampu mempelajari pola spasial gambar
Disaster Tweets	Teks pendek	TF-IDF + Logistic Regression	Konvensional	TF-IDF efektif untuk teks pendek dan noisy

Ringkasan Trade-off

Aspek	**Metode Konvensional**	**Deep Learning**
Akurasi	Kuat pada data kecil, tabular, dan teks pendek	Unggul pada data citra dalam eksperimen ini
Biaya Komputasi	Lebih ringan dan cepat dilatih	Lebih mahal dan membutuhkan waktu training lebih lama
Interpretabilitas	Lebih mudah dijelaskan	Lebih sulit diinterpretasikan
Kebutuhan Data	Dapat bekerja baik pada data terbatas	Umumnya membutuhkan data lebih besar
Feature Engineering	Membutuhkan fitur manual seperti HOG, PCA, TF-IDF	Dapat belajar fitur otomatis dari data
Risiko Overfitting	Relatif lebih rendah pada data kecil	Lebih tinggi jika data terbatas dan tuning kurang optimal

KESIMPULAN DAN REKOMENDASI

Berdasarkan eksperimen pada tiga kasus, dapat disimpulkan bahwa deep learning tidak selalu memberikan hasil terbaik pada semua jenis data. Performa model sangat dipengaruhi oleh karakteristik dataset, jumlah data, kompleksitas pola, serta biaya komputasi yang dibutuhkan. Pada kasus Titanic yang berbentuk data tabular, Logistic Regression menjadi model terbaik dengan accuracy 0.8436 dan F1-score 0.7971, mengungguli MLP yang memperoleh accuracy 0.7989 dan F1-score 0.7273. Hal ini menunjukkan bahwa pada dataset tabular kecil, metode konvensional masih lebih stabil, efisien, dan mudah diinterpretasikan dibanding deep learning.

Pada kasus Digit Recognizer, CNN memberikan performa terbaik dengan accuracy 0.9908 dan jumlah kesalahan paling sedikit, yaitu 77 gambar. Hasil ini menunjukkan bahwa deep learning memberikan peningkatan signifikan pada data citra karena CNN mampu mempelajari pola spasial secara otomatis, seperti garis, tepi, lengkungan, dan bentuk digit. Meskipun demikian, CNN membutuhkan waktu training paling lama, yaitu 309.4485 detik, sehingga terdapat trade-off antara peningkatan akurasi dan biaya komputasi.

Pada kasus Disaster Tweets, metode konvensional kembali menunjukkan performa terbaik. TF-IDF + Logistic Regression memperoleh accuracy 0.8188 dan F1-score 0.7723, sedangkan Embedding + LSTM hanya memperoleh accuracy 0.5706 dan F1-score 0.0000. Hasil ini menunjukkan bahwa untuk teks pendek dan noisy seperti tweet, representasi TF-IDF dengan classifier konvensional masih sangat kompetitif. LSTM belum mampu bekerja optimal karena tweet memiliki konteks yang terbatas, banyak noise, serta membutuhkan tuning dan data yang lebih besar agar performanya stabil.

Secara umum, deep learning paling efektif pada data citra, sedangkan metode konvensional masih sangat kompetitif pada data tabular kecil dan teks pendek. Oleh karena itu, pemilihan model tidak sebaiknya hanya didasarkan pada kompleksitas arsitektur, tetapi harus mempertimbangkan tipe data, ukuran dataset, kebutuhan interpretabilitas, waktu training, serta sumber daya komputasi yang tersedia.

Rekomendasi dari hasil eksperimen ini adalah sebagai berikut. Untuk dataset tabular kecil seperti Titanic, metode konvensional seperti Logistic Regression, Random Forest, atau model boosting sebaiknya dijadikan baseline utama sebelum menggunakan deep learning. Untuk dataset citra seperti Digit Recognizer, CNN direkomendasikan karena mampu mempelajari fitur visual secara otomatis dan memberikan peningkatan akurasi yang signifikan. Untuk dataset teks pendek seperti Disaster Tweets, pendekatan TF-IDF dengan Logistic Regression atau SVM tetap perlu dijadikan baseline kuat. Jika ingin menggunakan deep learning pada teks, diperlukan tuning tambahan seperti class weighting, threshold adjustment, penggunaan Bidirectional LSTM, atau embedding pretrained agar model dapat mempelajari representasi teks dengan lebih baik.

Dengan demikian, hasil eksperimen ini menegaskan bahwa metode terbaik sangat bergantung pada konteks data. Deep learning unggul ketika struktur data mendukung pembelajaran representasi otomatis, seperti pada gambar. Namun, metode konvensional tetap relevan dan bahkan dapat menjadi pilihan terbaik ketika dataset berukuran kecil, fitur sudah cukup informatif, atau interpretabilitas dan efisiensi menjadi prioritas.

DAFTAR PUSTAKA

Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M. A., Al-Amidie, M., & Farhan, L. (2021). Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, *8*(1). https://doi.org/10.1186/s40537-021-00444-8

Hochreiter, S., & Schmidhuber, J. (1997). LONG SHORT-TERM MEMORY. *Neural Computation*, *9(8)*. https://www.bioinf.jku.at/publications/older/2604.pdf

Kaggle. (n.d.-a). *Digit Recognizer*. Retrieved April 28, 2026, from https://www.kaggle.com/competitions/digit-recognizer/data

Kaggle. (n.d.-b). *Natural Language Processing with Disaster Tweets*. Retrieved April 28, 2026, from https://www.kaggle.com/competitions/nlp-getting-started/data

Kaggle. (n.d.-c). *Titanic - Machine Learning from Disaster*. Retrieved April 28, 2026, from https://www.kaggle.com/competitions/titanic/data

Pedregosa, F., Varoquaux, G., Gramfort, A., Vincent, M., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python Gaël Varoquaux Bertrand Thirion Vincent Dubourg Alexandre Passos PEDREGOSA, VAROQUAUX, GRAMFORT ET AL. Matthieu Perrot. In *Journal of Machine Learning Research* (Vol. 12). http://scikit-learn.sourceforge.net.

Stanford CS231. (2026). *CS231 Deep Learning for Computer Vision*. https://cs231n.github.io/convolutional-networks/

TensorFlow Core. (n.d.). *Keras: API tingkat tinggi untuk TensorFlow*. Retrieved April 28, 2026, from https://www.tensorflow.org/guide/keras?hl=id

LAMPIRAN

Lampiran A — Kasus 1: Titanic

```python
# ================================
# 1. Import Library
# ================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import random
import tensorflow as tf

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

Step 20 — Train-validation split

```python
from sklearn.model_selection import train_test_split

X_train, X_val, y_train, y_val = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=SEED,
    stratify=y
)

print("X_train:", X_train.shape)
print("X_val:", X_val.shape)
print("y_train:", y_train.shape)
print("y_val:", y_val.shape)
```

```
X_train: (712, 21)
X_val: (179, 21)
y_train: (712,)
y_val: (179,)
```

Bukti seed dan split data konsisten

Step 12 — Ekstraksi Title dari Name

```python
combined['Title'] = combined['Name'].str.extract(r' ([A-Za-z]+)\.', expand=False)

combined['Title'].value_counts()
```

Title	count
Mr	757
Miss	260
Mrs	197
Master	61
Rev	8
Dr	8
Col	4
Major	2
Mlle	2
Ms	2
Mme	1
Don	1
Sir	1
Lady	1
Capt	1
Countess	1
Jonkheer	1
Dona	1

Step 13 — FamilySize dan IsAlone

```python
combined['FamilySize'] = combined['SibSp'] + combined['Parch'] + 1

combined['IsAlone'] = 0
combined.loc[combined['FamilySize'] == 1, 'IsAlone'] = 1

combined[['SibSp', 'Parch', 'FamilySize', 'IsAlone']].head()
```

	SibSp	Parch	FamilySize	IsAlone
0	1	0	2	0
1	1	0	2	0
2	0	0	1	1
3	1	0	2	0
4	0	0	1	1

Preprocessing dan feature engineering utama

Step 14 — HasCabin

```python
combined['HasCabin'] = combined['Cabin'].notnull().astype(int)

combined[['Cabin', 'HasCabin']].head()
```

	Cabin	HasCabin
0	NaN	0
1	C85	1
2	NaN	0
3	C123	1
4	NaN	0

Step 16 — AgeBin dan FareBin

```python
combined['AgeBin'] = pd.cut(
    combined['Age'],
    bins=[0, 12, 19, 59, 100],
    labels=['Child', 'Teen', 'Adult', 'Senior']
)
```

Buat kategori harga tiket:

```python
combined['FareBin'] = pd.qcut(
    combined['Fare'],
    q=4,
    labels=['Low', 'Medium', 'High', 'VeryHigh']
)
```

Cek hasilnya:

```python
combined[['Age', 'AgeBin', 'Fare', 'FareBin']].head()
```

	Age	AgeBin	Fare	FareBin
0	22.0	Adult	7.2500	Low
1	38.0	Adult	71.2833	VeryHigh
2	26.0	Adult	7.9250	Medium
3	35.0	Adult	53.1000	VeryHigh
4	35.0	Adult	8.0500	Medium

Preprocessing dan feature engineering utama

Step 32 — Buat model MLP

```python
input_dim = X_train_scaled.shape[1]

mlp = Sequential([
    Dense(
        64,
        activation='relu',
        input_shape=(input_dim,),
        kernel_regularizer=l2(0.001)
    ),
    BatchNormalization(),
    Dropout(0.3),

    Dense(
        32,
        activation='relu',
        kernel_regularizer=l2(0.001)
    ),
    BatchNormalization(),
    Dropout(0.2),

    Dense(16, activation='relu'),

    Dense(1, activation='sigmoid')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: U
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Step 34 — Tampilkan summary model

+ Kode + Teks

```python
mlp.summary()
```

Tambahkan sel kode Ctrl+M B

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	1,408
batch_normalization (BatchNormalization)	(None, 64)	256
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
batch_normalization_1 (BatchNormalization)	(None, 32)	128
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

```
Total params: 4,417 (17.25 KB)
Trainable params: 4,225 (16.50 KB)
Non-trainable params: 192 (768.00 B)
```

Arsitektur MLP dan jumlah parameter

Step 41 — Tambahkan MLP ke tabel hasil

```python
results.append(result_mlp)

results_df = pd.DataFrame(results)
results_df.sort_values(by='F1-score', ascending=False)
```

	Model	Accuracy	Precision	Recall	F1-score	Training Time
0	Logistic Regression	0.843575	0.797101	0.797101	0.797101	0.123542
1	Random Forest	0.798883	0.720000	0.782609	0.750000	0.592791
2	MLP Deep Learning	0.798883	0.761905	0.695652	0.727273	16.138709

Tabel hasil akhir Logistic Regression, Random Forest, dan MLP

Step 51 — Ambil 3 contoh error untuk laporan

```
error_examples = common_errors[[
    'PassengerId', 'Name', 'Sex', 'Age', 'Pclass', 'SibSp', 'Parch',
    'Fare', 'Embarked', 'Actual', 'Pred_LogReg', 'Pred_RF', 'Pred_MLP'
]].head(3)

error_examples
```

	PassengerId	Name	Sex	Age	Pclass	SibSp	Parch	Fare	Embarked	Actual	Pred_LogReg	Pred_RF
553	554	Leeni, Mr. Fahim ("Philip Zenni")	male	22.0	3	0	0	7.225	C	1	0	0
536	537	Butt, Major. Archibald Willingham	male	45.0	1	0	0	26.550	S	0	1	1
712	713	Taylor, Mr. Elmer Zebley	male	48.0	1	1	0	52.000	S	1	0	0

Contoh minimal 3 salah prediksi

Lampiran B — Digit Recognizer

```
import random
import tensorflow as tf

SEED = 42

np.random.seed(SEED)
random.seed(SEED)
tf.random.set_seed(SEED)
```

5. Train-Validation Split

```
X_train, X_val, y_train, y_val = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=SEED,
    stratify=y
)

print("X_train:", X_train.shape)
print("X_val:", X_val.shape)
print("y_train:", y_train.shape)
print("y_val:", y_val.shape)
```

```
X_train: (33600, 784)
X_val: (8400, 784)
y_train: (33600,)
y_val: (8400,)
```

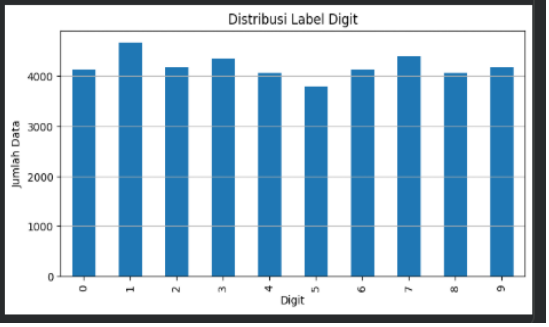
Bukti seed dan split data konsisten

```python
train_df["label"].value_counts().sort_index()
```

label	count
0	4132
1	4684
2	4177
3	4351
4	4072
5	3795
6	4137
7	4401
8	4063
9	4188

Visualisasi distribusi label:

```python
plt.figure(figsize=(8, 4))
train_df["label"].value_counts().sort_index().plot(kind="bar")
plt.title("Distribusi Label Digit")
plt.xlabel("Digit")
plt.ylabel("Jumlah Data")
plt.grid(axis="y")
plt.show()
```



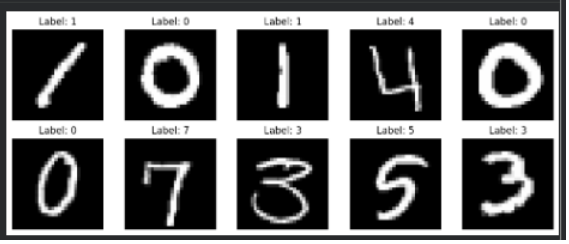
Visualisasi contoh gambar:

```python
X_preview = train_df.drop("label", axis=1).values
y_preview = train_df["label"].values

plt.figure(figsize=(10, 4))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_preview[i].reshape(28, 28), cmap="gray")
    plt.title(f"Label: {y_preview[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



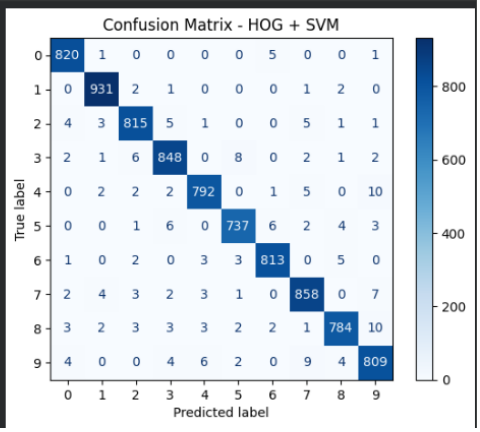
EDA distribusi digit dan contoh gambar

9. Confusion Matrix

```python
cm_hog_svm = confusion_matrix(y_val, y_pred_hog_svm)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_hog_svm,
    display_labels=np.arange(10)
)

disp.plot(cmap="Blues", values_format="d")
plt.title("Confusion Matrix - HOG + SVM")
plt.show()
```

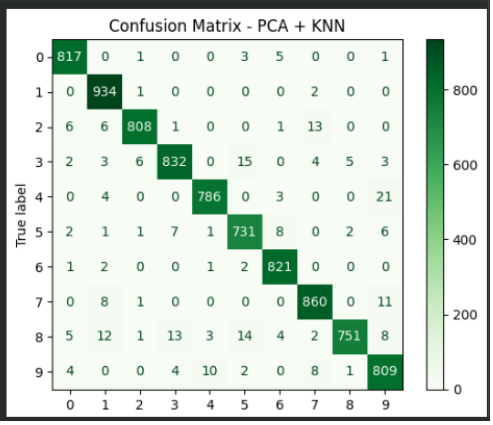


7. Confusion Matrix

```python
cm_pca_knn = confusion_matrix(y_val, y_pred_pca_knn)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_pca_knn,
    display_labels=np.arange(10)
)

disp.plot(cmap="Greens", values_format="d")
plt.title("Confusion Matrix - PCA + KNN")
plt.show()
```



Metode konvensional HOG+SVM dan PCA+KNN

4. Build Model CNN

```python
cnn_model = Sequential([
    Conv2D(
        filters=32,
        kernel_size=(3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(
        filters=64,
        kernel_size=(3, 3),
        activation="relu"
    ),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),

    Dense(128, activation="relu"),
    Dropout(0.3),

    Dense(10, activation="softmax")
])

cnn_model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_co
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)
```

Arsitektur CNN dan jumlah parameter

Urutkan berdasarkan akurasi:

```python
final_comparison.sort_values(by="Accuracy", ascending=False)
```

	Model	Accuracy	Training Time	Inference Time	Misclassification Count
2	CNN	0.990833	309.448498	2.255881	77
0	HOG + SVM	0.977024	13.908682	0.042635	193
1	PCA + KNN	0.970119	1.182801	3.073194	251

Tabel hasil akhir dan contoh salah klasifikasi

Lampiran C — Disaster Tweets

```python
import pandas as pd
import numpy as np
import random
import re
import time

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,
precision_score, recall_score, f1_score, classification_report, confusion_matrix

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC

import matplotlib.pyplot as plt
import seaborn as sns

import tensorflow as tf
```

```python
SEED = 42

random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

5. Split Data Train dan Validation

```python
X = train_df['clean_text']
y = train_df['target']

X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=0.2,
    random_state=SEED,
    stratify=y
)
```

Bukti seed dan split data konsisten

3. Exploratory Data Analysis Sederhana

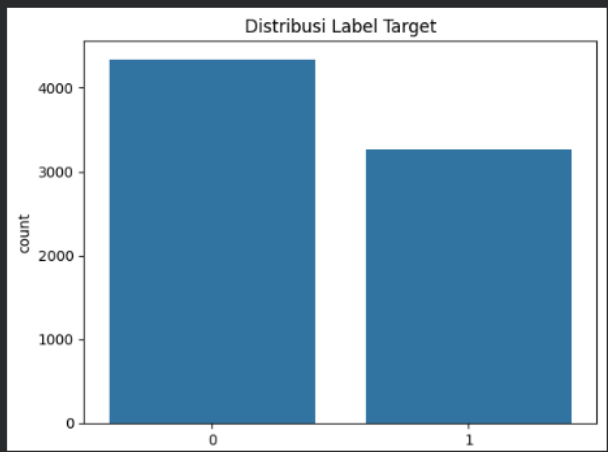
```
train_df['target'].value_counts()
```

target	count
0	4342
1	3271

dtype: int64

Visualisasi distribusi label:

```
sns.countplot(data=train_df, x='target')
plt.title('Distribusi Label Target')
plt.show()
```



EDA distribusi target

4. Text Preprocessing

Preprocessing yang dilakukan:

- Lowercasing
- Menghapus URL
- Menghapus mention
- Menghapus hashtag symbol #, tapi isi katanya tetap dipakai
- Menghapus karakter non-huruf
- Menghapus spasi berlebih

```python
def clean_text(text):
    text = text.lower()
    text = re.sub(r'http\S+|www\S+|https\S+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#', '', text)
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

6. Baseline 1 — TF-IDF Unigram + LogistiRegression

6.1 TF-IDF Unigram

```python
tfidf_uni = TfidfVectorizer(
    ngram_range=(1, 1),
    max_features=10000
)

X_train_uni = tfidf_uni.fit_transform(X_train)
X_val_uni = tfidf_uni.transform(X_val)
```

7. Baseline 2 — TF-IDF Unigram + Bigram + Linear SVM

7.1 TF-IDF Unigram + Bigram

```python
tfidf_bi = TfidfVectorizer(
    ngram_range=(1, 2),
    max_features=15000
)

X_train_bi = tfidf_bi.fit_transform(X_train)
X_val_bi = tfidf_bi.transform(X_val)
```

Preprocessing teks dan TF-IDF

```python
# Tuning sederhana Logistic Regression

for C in [0.1, 1, 10]:
    model = LogisticRegression(max_iter=1000, C=C, random_state=S
    model.fit(X_train_uni, y_train)
    pred = model.predict(X_val_uni)
    print("C:", C, "F1:", f1_score(y_val, pred))
```

```
C: 0.1 F1: 0.6429277942631059
C: 1 F1: 0.7722772277227723
C: 10 F1: 0.7627251370399374
```

```python
lr_model = LogisticRegression(max_iter=1000, C=1, random_state=SE
lr_model.fit(X_train_uni, y_train)
```

```
LogisticRegression
LogisticRegression(C=1, max_iter=1000, random_state=42)
```

```python
# Tuning sederhana Linear SVM

for C in [0.1, 1, 10]:
    model = LinearSVC(C=C)
    model.fit(X_train_bi, y_train)
    pred = model.predict(X_val_bi)
    print("C:", C, "F1:", f1_score(y_val, pred))
```

```
C: 0.1 F1: 0.7636669470142977
C: 1 F1: 0.759592795614722
C: 10 F1: 0.7293404094010614
```

```python
svm_model = LinearSVC(C=0.1)
svm_model.fit(X_train_bi, y_train)
```

```
LinearSVC
LinearSVC(C=0.1)
```

```python
# Tuning sederhana Multinomial Naive Bayes

for alpha in [0.1, 0.5, 1.0]:
    model = MultinomialNB(alpha=alpha)
    model.fit(X_train_bi, y_train)
    pred = model.predict(X_val_bi)
    print("alpha:", alpha, "F1:", f1_score(y_val, pred))
```

```
alpha: 0.1 F1: 0.7562604340567612
alpha: 0.5 F1: 0.76
alpha: 1.0 F1: 0.7511071744906997
```

```python
nb_model = MultinomialNB(alpha=0.5)
nb_model.fit(X_train_bi, y_train)
```

```
MultinomialNB
MultinomialNB(alpha=0.5)
```

Model konvensional dan tuning sederhana

11. Tokenization dan Padding

```python
max_words = 10000
max_len = 50

tokenizer = Tokenizer(num_words=max_words, oov_token='<OOV>')
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_val_seq = tokenizer.texts_to_sequences(X_val)

X_train_pad = pad_sequences(X_train_seq, maxlen=max_len, padding='post', truncating='post')
X_val_pad = pad_sequences(X_val_seq, maxlen=max_len, padding='post', truncating='post')
```

Arsitektur LSTM dan jumlah parameter

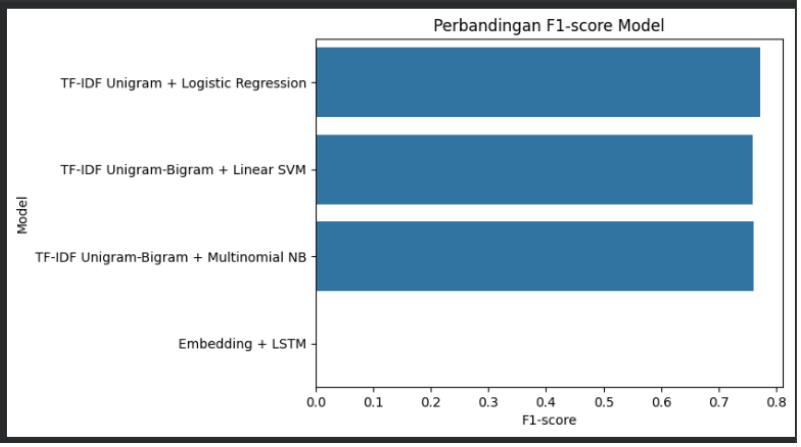
Urutkan berdasarkan F1-score:

```python
results.sort_values(by='F1-score', ascending=False)
```

	Model	Accuracy	Precision	Recall	F1-score	Training Time	Inference Time
0	TF-IDF Unigram + Logistic Regression	0.818779	0.838710	0.715596	0.772277	0.068742	0.003182
2	TF-IDF Unigram-Bigram + Multinomial NB	0.818779	0.881048	0.668196	0.760000	0.007422	0.002229
1	TF-IDF Unigram-Bigram + Linear SVM	0.798424	0.778491	0.741590	0.759593	0.038291	0.002149
3	Embedding + LSTM	0.570584	0.000000	0.000000	0.000000	109.538740	0.951490

Visualisasi F1-score:

```python
sns.barplot(data=results, x='F1-score', y='Model')
plt.title('Perbandingan F1-score Model')
plt.show()
```



Tabel hasil akhir

18. Error Analysis Logistic Regression

```python
error_lr = pd.DataFrame({
    'text': X_val.values,
    'actual': y_val.values,
    'predicted': y_pred_lr
})

error_lr = error_lr[error_lr['actual'] != error_lr['predicted']]
error_lr.head(10)
```

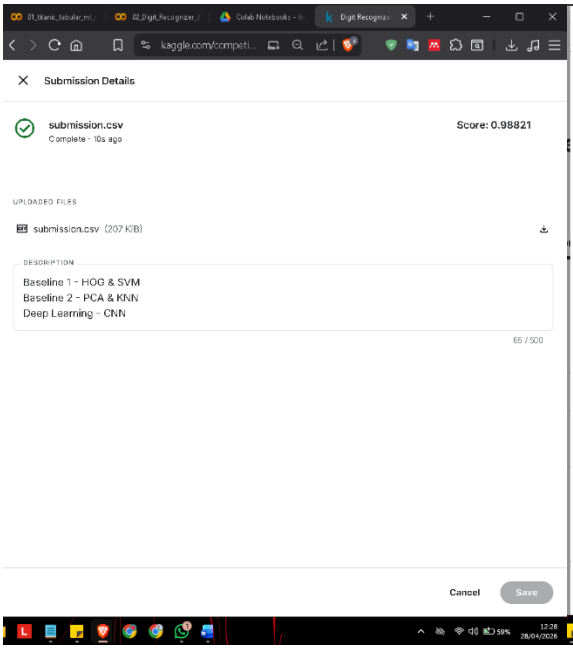
	text	actual	predicted
5	hollywood movie about trapped miners released ...	1	0
6	thu aug gmt utc millcityio theramin sirens	1	0
7	i cant drown my demons they know how to swim	1	0
13	we need these plants in the pacific during the...	1	0
14	if firefighters acted like cops theyd drive ar...	0	1
16	it looks like a mudslide poor thing greatbriti...	1	0
22	on the flip side im at walmart and there is a ...	1	0
24	maybe we should call israel and tell them were...	0	1
27	i agree with certain cultural appropriation th...	1	0
29	the dress memes have officially exploded on th...	1	0

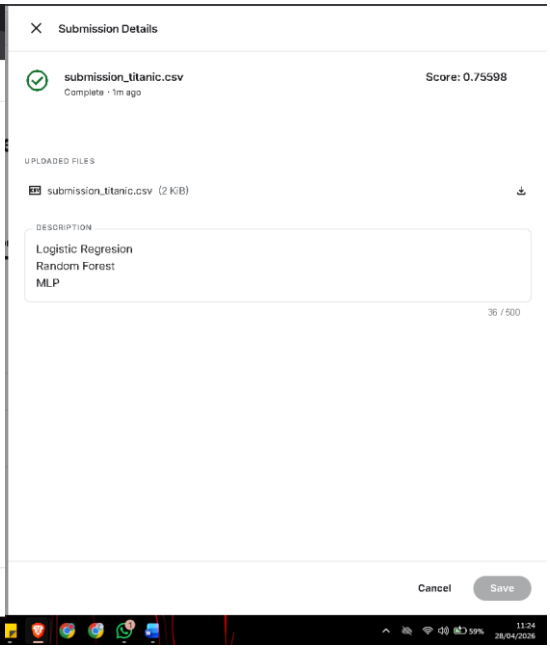
Tampilkan contoh salah klasifikasi:

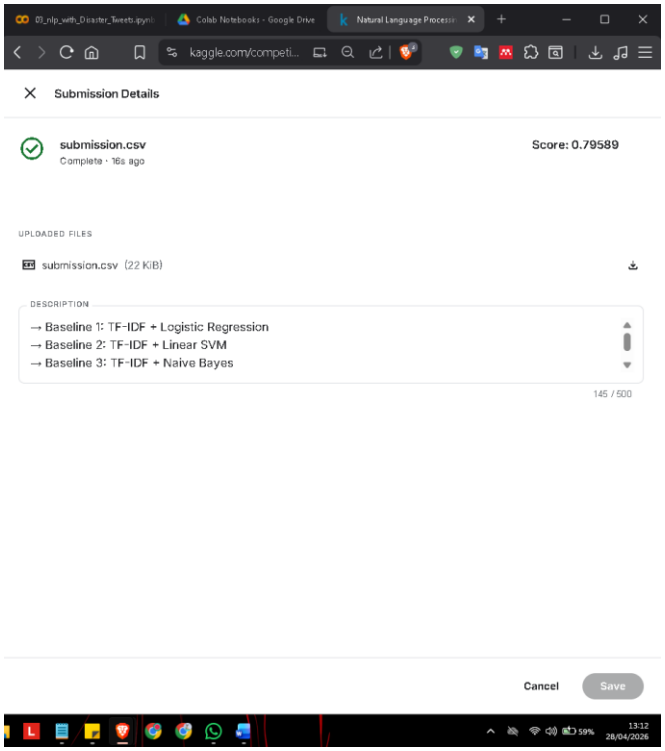
```python
for i, row in error_lr.head(5).iterrows():
    print("Tweet:", row['text'])
    print("Actual:", row['actual'])
    print("Predicted:", row['predicted'])
    print("-" * 80)
```

```
Tweet: hollywood movie about trapped miners released in chile the hollywood movie about trapped miners starring
Actual: 1
Predicted: 0
--------------------------------------------------------------------------------
Tweet: thu aug gmt utc millcityio theramin sirens
Actual: 1
Predicted: 0
--------------------------------------------------------------------------------
Tweet: i cant drown my demons they know how to swim
Actual: 1
Predicted: 0
--------------------------------------------------------------------------------
Tweet: we need these plants in the pacific during the cyclone seasons it would help
Actual: 1
Predicted: 0
--------------------------------------------------------------------------------
Tweet: if firefighters acted like cops theyd drive around shooting a flamethrower at burning buildings
Actual: 0
Predicted: 1
```

Error analysis







Submission Kaggle per Kasus